

数据融合与智能分析实验报告 5-6

班级	物联网 2202	学号	223428010210
专业	物联网工程	姓名	陈梓欣
实验成绩			
指导教师	郭振洲	2025 年 6 月 24 日	

一、实验题目：金融数据综合练习与 Logistic 回归及实现

二、实验地点：机械 307-6，雨课堂

三、实验目的：

- 1 理解金融数据特点，学会 resample 方法进行不同的周期统计；
- 2 理解 python 时间对象的基础定义和操作；
- 3 完成金融数据的综合练习；
- 4 理解梯度相关概念和含义，理解梯度下降法原理及应用背景；
- 5 理解 sigmoid 函数及其意义；
- 6 理解 Logistic 回归相关原理，应用梯度上升方法求回归问题目标函数的近似值；

似值；

四、实验内容（完成题目代码及运行结果，代码注释中包含个人信息）

1. 读取 stock.xlsx 文件中的股票信息作为一个 DataFrame，计算每天最高价与最低价的算术平均值。将此平均值作为单独的一列添加到原始数据的 DataFrame 对象中。

```
# 223428010210 陈梓欣
'''读取stock.xlsx文件中的股票信息作为一个DataFrame，
计算每天最高价与最低价的算术平均值。
将此平均值作为单独的一列添加到原始数据的DataFrame对象中。
'''

# coding=utf-8
import pandas as pd
df=pd.read_excel('stock.xlsx',sheet_name='stock',
                 index_col='Date',usecols='A:E',nrows=5)
print('原始DataFrame对象: \n',df)
#计算每天最高价与最低价的算术平均值
#result=(df.High+df.Low) / 2
result=(df['High']+df['Low']) / 2
df['mean']=result
print('添加平均价格后的DataFrame: \n',df)
```

原始DataFrame对象:

	Open	High	Low	Close
Date				
2018-04-02	9.99	10.14	9.51	9.53
2018-04-03	9.63	9.77	9.30	9.55
2018-04-04	9.08	9.81	9.04	9.77
2018-04-05	10.05	10.20	9.91	10.02
2018-04-06	9.83	10.10	9.50	9.61

添加平均价格后的DataFrame:

	Open	High	Low	Close	mean
Date					
2018-04-02	9.99	10.14	9.51	9.53	9.825
2018-04-03	9.63	9.77	9.30	9.55	9.535
2018-04-04	9.08	9.81	9.04	9.77	9.425
2018-04-05	10.05	10.20	9.91	10.02	10.055
2018-04-06	9.83	10.10	9.50	9.61	9.800

2. Pandas 中的 Timestamp 类的基础操作。

```
# 223428010210 陈梓欣
# Pandas 中的Timestamp类将numpy中datetime64数组的高效性与datetime和dateutil的易用性相结合，来处理日期和时间
import numpy as np
import pandas as pd
#获取当前时间
dt1 = pd.Timestamp.now()
print(dt1)
#用字符串初始化时间对象
dt2 = pd.Timestamp("2024-10-1")
print(dt2)
dt3 = pd.to_datetime("2024-10-3 13:55")
print(dt3)
#返回星期几的英文
print(dt1.strftime("%A"))
#时间对象运算
print(dt1 - dt3)
print(dt1 + pd.Timedelta(days=2))
print(dt1 + 5 * pd.Timedelta(days=2))
print(dt1 - 5 * pd.Timedelta(days=2))
```

```
2025-06-24 14:28:43.778945
2024-10-01 00:00:00
2024-10-03 13:55:00
Tuesday
264 days 00:33:43.778945
2025-06-26 14:28:43.778945
2025-07-04 14:28:43.778945
2025-06-14 14:28:43.778945
```

```
# 223428010210 陈梓欣
#时间作为行或列的标签
#通过Timestamp对象的向量化计算得到DatetimeIndex对象dIndex也可以作为行或列的标签
import numpy as np
import pandas as pd
#建立一个时间范围
dt1 = pd.Timestamp.now()
dIndex = dt1 + pd.to_timedelta(np.arange(5), "D")
print(dIndex)
s1 = pd.Series(range(5), index=dIndex)
print(s1)
dateRange = pd.date_range(dt1, periods=3)
print(dateRange)
df1 = pd.DataFrame(np.arange(9).reshape(3,3), columns=dateRange)
print(df1)

DatetimeIndex(['2025-06-24 14:32:41.166048', '2025-06-25 14:32:41.166048',
              '2025-06-26 14:32:41.166048', '2025-06-27 14:32:41.166048',
              '2025-06-28 14:32:41.166048'],
              dtype='datetime64[ns]', freq=None)
2025-06-24 14:32:41.166048    0
2025-06-25 14:32:41.166048    1
2025-06-26 14:32:41.166048    2
2025-06-27 14:32:41.166048    3
2025-06-28 14:32:41.166048    4
dtype: int64
DatetimeIndex(['2025-06-24 14:32:41.166048', '2025-06-25 14:32:41.166048',
              '2025-06-26 14:32:41.166048'],
              dtype='datetime64[ns]', freq='D')
2025-06-24 14:32:41.166048    2025-06-25 14:32:41.166048  \
0                                0                                1
1                                3                                4
2                                6                                7

2025-06-26 14:32:41.166048
0                                2
1                                5
2                                8
```

3. 读取文件 multi_stock.xlsx 中的股票信息数据，以日期为索引，分别计算每年的平均价格、每周的价格，并画出相应的折线图。

```
# 223428010210 陈梓欣
# 重新采样
# resample() 返回一个pandas.core.resample.DatetimeIndexResampler
# DatetimeIndexResampler对象里面包含的是分组后的详细数据
# 需要添加统计方法（如mean、sum等）计算后得到DataFrame

#以年统计均值resample("Y")
data2 = data.resample("Y").mean()
print("以年为时间段进行采样，并取均值：\n", data2[:5])

# 创建图
plt.figure(figsize=(10,7)) #设置图像大小

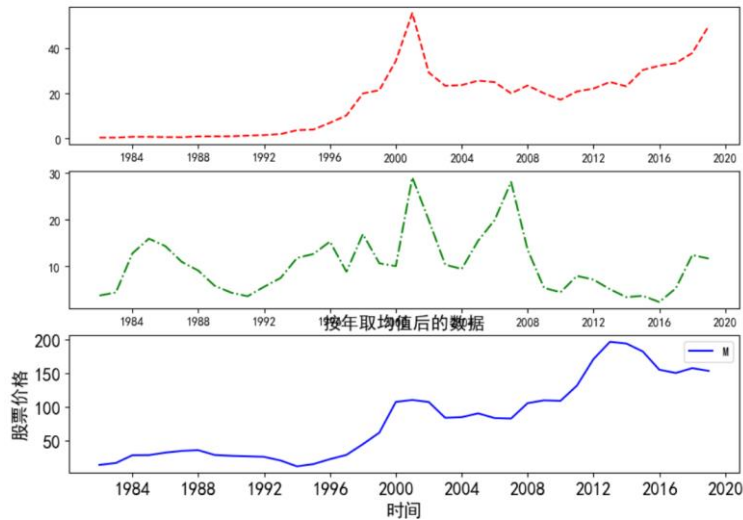
ax1 = plt.subplot(3,1,1)
ax2 = plt.subplot(3,1,2)
ax3 = plt.subplot(3,1,3)

plt.sca(ax1) # 选择子图
plt.plot(data2.index, data2["C"], "r--", label="C")
plt.sca(ax2)
plt.plot(data2.index, data2["D"], "g-.", label="D")
plt.sca(ax3)
plt.plot(data2.index, data2["M"], "b-", label="M")
# 设置刻度字体大小
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)
plt.title("按年取均值后的数据", fontsize=15)
plt.xlabel("时间", fontsize=15)
plt.ylabel("股票价格", fontsize=15)
plt.legend()
```

```

以年为时间段进行采样，并取均值：
      C          D          M
date
1981-12-31  0.338387   3.724391  14.497406
1982-12-31  0.328274   4.371089  17.299037
1983-12-31  0.681165  12.751976  28.771616
1984-12-31  0.676836  15.925642  28.949234
1985-12-31  0.567884  14.329117  32.601562
C:\Users\SHI_Z\AppData\Local\Temp\ipykernel_28276\4118482643.py:8: FutureWarning: 'Y' is deprecated and
will be removed in a future version, please use 'YE' instead.
data2 = data.resample("Y").mean()
<matplotlib.legend.Legend at 0x1f822790980>

```



```

# 223428010210 陈梓欣
#asfreq()方法返回符合指定频率的新索引的原始数据 data.resample("W-MON")
#以星期为单位统计数据，重采样
data3 = data.resample("W-MON").asfreq()
print("取每周一的数据：\n",data3[:5])

# 创建图
plt.figure(figsize=(10,7)) #设置图像大小

ax1 = plt.subplot(3,1,1)
ax2 = plt.subplot(3,1,2)
ax3 = plt.subplot(3,1,3)

plt.sca(ax1) # 选择子图
plt.plot(data3.index, data3["C"], "r--", label="C")
plt.sca(ax2)
plt.plot(data3.index, data3["D"], "g-.", label="D")
plt.sca(ax3)
plt.plot(data3.index, data3["M"], "b-", label="M")
# 设置刻度字体大小
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)
plt.title("每周一的数据", fontsize=15)
plt.xlabel("时间", fontsize=15)
plt.ylabel("股票价格", fontsize=15)
plt.legend()

#调整子图间距: wspace 调整横向距离, hspace 调整纵向距离
plt.subplots_adjust(hspace=0.8)
plt.show()

```

取每周一的数据：

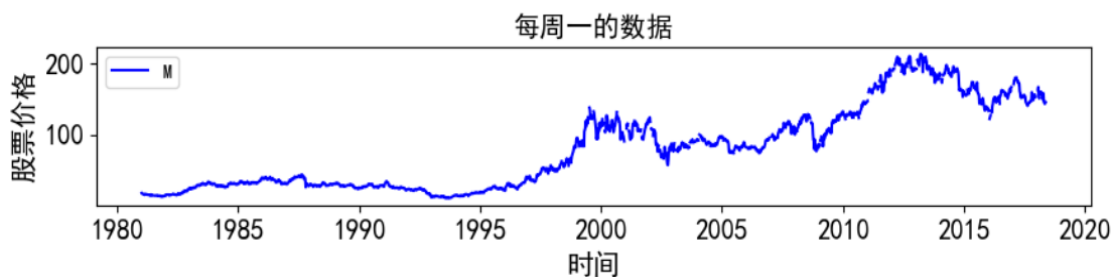
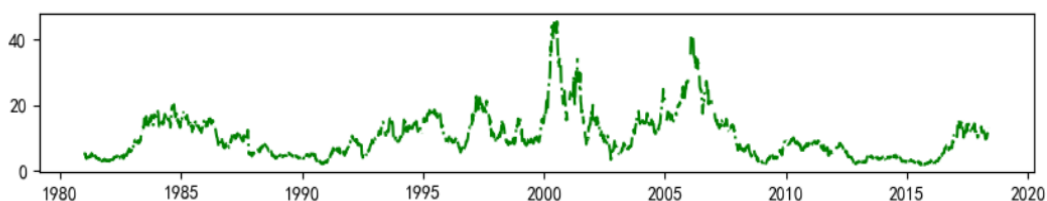
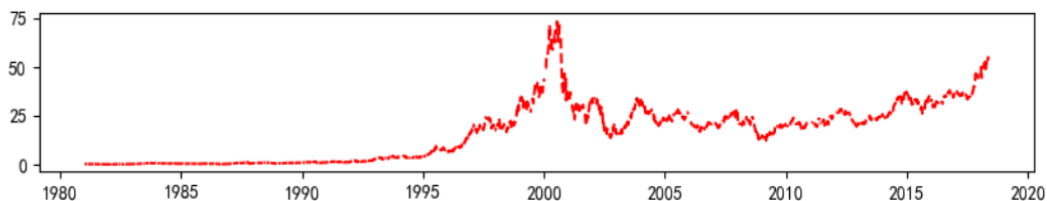
```

      C          D          M
date
1981-01-05  0.434896  5.437500  17.56250
1981-01-12  0.395833  4.666667  16.68750
1981-01-19  0.416667  4.812500  16.65625
1981-01-26  0.398438  4.083333  16.25000
1981-02-02  0.375000  3.479167  15.84375

```

取每周二的数据:

	C	D	M
date			
1981-01-05	0.434896	5.437500	17.56250
1981-01-12	0.395833	4.666667	16.68750
1981-01-19	0.416667	4.812500	16.65625
1981-01-26	0.398438	4.083333	16.25000
1981-02-02	0.375000	3.479167	15.84375



4. 使用 tushare 包获取 600519 股票的历史行情数据。

(1) 输出该股票所有收盘比开盘上涨 3% 以上的日期。

```
# 223428010210 陈梓欣
#输出该股票所有收盘比开盘上涨3%以上的日期
#伪代码: (收盘-开盘)/开盘 > 0.03
(df['close'] - df['open']) / df['open'] > 0.03

#在分析的过程中如果产生了boolean值则下一步马上将布尔值作为源数据的行索引
#如果布尔值作为df的行索引, 则可以取出true对应的行数据, 忽略false对应的行数据
df.loc[(df['close'] - df['open']) / df['open'] > 0.03] #获取了True对应的行数据 (满足需求的行数据)

df.loc[(df['close'] - df['open']) / df['open'] > 0.03].index #df的行数据

DatetimeIndex(['2001-08-27', '2001-08-28', '2001-09-10', '2001-12-21',
               '2002-01-18', '2002-01-31', '2003-01-14', '2003-10-29',
               '2004-01-05', '2004-01-14',
               ...,
               '2024-03-12', '2024-06-24', '2024-07-02', '2024-07-31',
               '2024-09-24', '2024-09-26', '2024-09-30', '2024-10-18',
               '2024-11-07', '2025-03-14'],
              dtype='datetime64[ns]', name='trade_date', length=363, freq=None)
```

(2) 输出该股票所有开盘比前日收盘跌幅超过 2% 的日期。

```
# 223428010210 陈梓欣
#输出该股票所有开盘比前日收盘跌幅超过2%的日期
#伪代码: (开盘-前日收盘)/前日收盘 < -0.02
(df['open'] - df['close'].shift(1))/df['close'].shift(1) < -0.02
#将布尔值作为源数据的行索引取出True对应的行数据
df.loc[(df['open'] - df['close'].shift(1))/df['close'].shift(1) < -0.02]

df.loc[(df['open'] - df['close'].shift(1))/df['close'].shift(1) < -0.02].index

DatetimeIndex(['2001-09-12', '2002-06-26', '2002-07-25', '2002-12-13',
               '2003-07-14', '2004-07-01', '2004-10-29', '2005-08-05',
               '2006-05-25', '2006-08-21',
               ...,
               '2021-11-01', '2022-03-14', '2022-03-15', '2022-03-28',
               '2022-10-13', '2022-10-24', '2022-11-28', '2023-02-06',
               '2024-06-24', '2025-04-07'],
              dtype='datetime64[ns]', name='trade_date', length=107, freq=None)
```

(3) 假如从 2010 年 1 月 1 日开始, 每月第一个交易日买入 1 手股票, 每年最后一个交易日卖出所有股票, 到今天为止收益如何?

```
# 223428010210 陈梓欣
#交易时间范围
new_df = df['2010-01': '2025-05']
new_df
```

	ts_code	open	high	low	close	pre_close	change	pct_chg	vol	am
trade_date										
2010-01-04	600519.SH	172.00	172.00	169.31	169.94	169.82	0.12	0.0700	44304.88	75340
2010-01-05	600519.SH	170.99	171.50	169.00	169.44	169.94	-0.50	-0.2900	31513.18	53572
2010-01-06	600519.SH	168.99	169.50	166.31	166.76	169.44	-2.68	-1.5800	39889.03	66607
2010-01-07	600519.SH	166.76	167.19	161.88	163.72	166.76	-3.04	-1.8200	48825.55	80144
2010-01-08	600519.SH	164.00	164.00	160.10	162.00	163.72	-1.72	-1.0500	36702.09	59316

```
# 223428010210 陈梓欣
#买股票: 找每个月的第一个交易日对应的行数据(捕获到开盘价) ==》每月的第一行数据
#根据月份从原始数据中提取指定的数据
#每月第一个交易日对应的行数据
df_monthly = new_df.resample('M').first()#数据的重新取样
df_monthly
```

```
C:\Users\SHI_Z\AppData\Local\Temp\ipykernel_28276\3085311904.py:5: FutureWarning: 'M' is deprecated
and will be removed in a future version, please use 'ME' instead.
df_monthly = new_df.resample('M').first()#数据的重新取样
```

	ts_code	open	high	low	close	pre_close	change	pct_chg	vol	am
trade_date										
2010-01-31	600519.SH	172.00	172.00	169.31	169.94	169.82	0.12	0.0700	44304.88	75340
2010-02-28	600519.SH	168.88	169.58	167.01	168.89	168.54	0.35	0.2100	29655.94	49986
2010-03-31	600519.SH	166.45	167.45	165.99	166.24	166.84	-0.60	-0.3600	21734.74	36155

```
# 223428010210 陈梓欣
#买入股票花费的总金额
cost = df_monthly['open'].sum()*100
cost
```

15229505.000000002

```
# 223428010210 陈梓欣
#卖出股票到手的钱
#特殊情况：2025年买入的股票卖不出去
new_df.resample('A').last()
#将2020年最后一行切出去
df_yearly = new_df.resample('A').last()[::-1]
df_yearly
```

C:\Users\SHI_Z\AppData\Local\Temp\ipykernel_28276\3755523756.py:4: FutureWarning: 'A' is deprecated and will be removed in a future version, please use 'YE' instead.
new_df.resample('A').last()
C:\Users\SHI_Z\AppData\Local\Temp\ipykernel_28276\3755523756.py:6: FutureWarning: 'A' is deprecated and will be removed in a future version, please use 'YE' instead.
df_yearly = new_df.resample('A').last()[::-1]

	ts_code	open	high	low	close	pre_close	change	pct_chg	vol	am
trade_date										
2010-12-31	600519.SH	181.80	184.28	181.05	183.92	181.93	1.99	1.0900	46084.87	84307
2011-12-31	600519.SH	192.70	194.88	190.00	193.30	192.21	1.09	0.5700	29460.86	56987
2012-12-31	600519.SH	213.31	214.80	206.35	209.02	212.30	-3.28	-1.5500	51914.83	108577

```
# 223428010210 陈梓欣
#卖出股票到手的钱
resv = df_yearly['open'].sum()*1200
resv
```

```
# 223428010210 陈梓欣
#最后手中剩余的股票需要估量其价值计算到总收益中
#使用昨天的收盘价作为剩余股票的单价
#1月到6月持600
last_monry = 600*new_df['close'][-1]
last_monry
```

C:\Users\SHI_Z\AppData\Local\Temp\ipykernel_28276\1894266965.py:5: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
last_monry = 600*new_df['close'][-1]

913200.0

```
# 223428010210 陈梓欣
#计算总收益
resv+last_monry-cost
```

1162422.9999999981

5. 简要说明梯度下降法的迭代过程。

梯度下降法是一种常用的优化算法，主要用于最小化目标函数。其核心思想是：**沿着函数梯度的反方向不断迭代，逐步逼近最小值点。**

迭代过程步骤如下：

1. **初始化参数：** 随机选择一个初始点（即参数值）。
2. **计算梯度：** 在当前参数点，计算目标函数对各参数的偏导数（梯度）。
3. **更新参数：** 根据公式

$$\theta = \theta - \alpha \cdot \nabla J(\theta)$$

其中:

- θ 表示参数
- α 是学习率 (步长)
- $\nabla J(\theta)$ 是目标函数在当前点的梯度

4. **重复迭代**: 重复步骤2和3, 直到满足停止条件, 如梯度趋于0或达到最大迭代次数。

通过不断迭代, 参数会逐步逼近使目标函数最小的值。梯度下降法适用于线性回归、逻辑回归、神经网络等多种机器学习模型的训练过程。

6. 配置数据文件 LogiReg_data.txt, 并实现如下操作:

(1) 读取文件, 实现对数据预览:

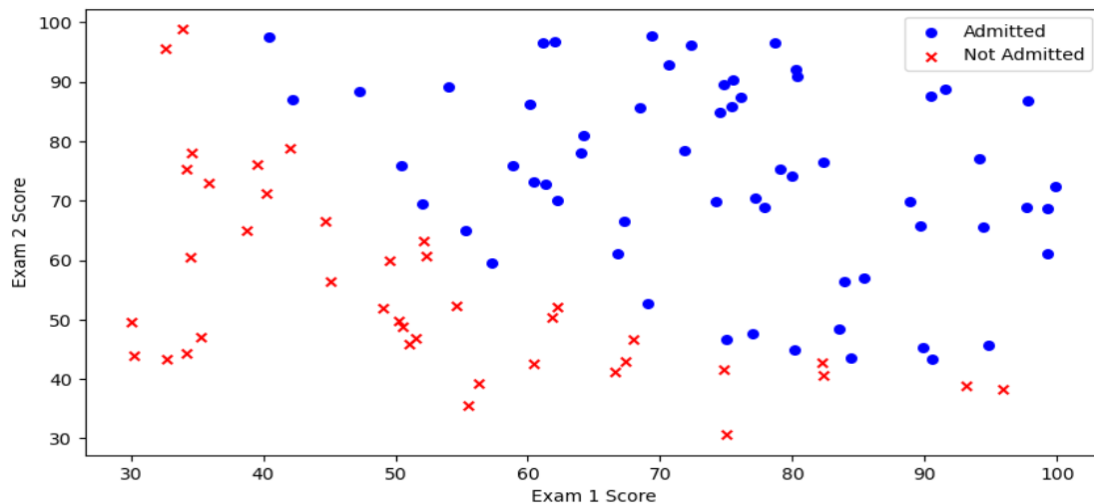
```
# 223428010210 陈梓欣
#读取数据并预览
import os
path = './LogiReg_data.txt'
# './LogiReg_data.txt'
pdData = pd.read_csv(path, header=None, names=['Exam 1', 'Exam 2', 'Admitted'])
pdData.head()
```

	Exam 1	Exam 2	Admitted
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1

(2) 查看数据中正负样本的比例绘制散点图:

```
# 223428010210 陈梓欣
#指定正例负例
positive = pdData[pdData['Admitted'] == 1]
negative = pdData[pdData['Admitted'] == 0]

fig, ax = plt.subplots(figsize=(10,5))
ax.scatter(positive['Exam 1'], positive['Exam 2'], s=30, c='b', marker='o', label='Admitted')
ax.scatter(negative['Exam 1'], negative['Exam 2'], s=30, c='r', marker='x', label='Not Admitted')
ax.legend()
ax.set_xlabel('Exam 1 Score')
ax.set_ylabel('Exam 2 Score')
plt.show()
```



7. 自定义 Logistic 回归过程:

(1) 定义 sigmoid 函数:

```
# 223428010210 陈梓欣
#1sigmoid 函数
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

(2) 定义预测函数:

```
# 223428010210 陈梓欣
#2 预测函数h(x)
def model(X, theta):

    return sigmoid(np.dot(X, theta.T))
```

(3) 定义目标函数(损失函数):

```
# 223428010210 陈梓欣
#3 损失函数
def cost(X, y, theta):
    left = np.multiply(-y, np.log(model(X, theta)))
    right = np.multiply(1 - y, np.log(1 - model(X, theta)))
    return np.sum(left - right) / (len(X))
```

```
cost(X, y, theta)
```

```
0.6931471805599453
```

(4) 定义求梯度函数:

```
# 223428010210 陈梓欣
#梯度下降法
#求梯度
def gradient(X, y, theta):
    grad = np.zeros(theta.shape)
    #误差项
    error = (model(X, theta) - y).ravel()
    for j in range(len(theta.ravel())): #for each parameter
        term = np.multiply(error, X[:,j])
        grad[0, j] = np.sum(term) / len(X)

    return grad
```

(5) 定义梯度下降过程的终止策略函数:

```
# 223428010210 陈梓欣
#迭代停止策略 次数 值 梯度
STOP_ITER = 0
STOP_COST = 1
STOP_GRAD = 2

def stopCriterion(type, value, threshold):
    #设定三种不同的停止策略
    if type == STOP_ITER:
        return value > threshold
    elif type == STOP_COST:
        return abs(value[-1]-value[-2]) < threshold
    elif type == STOP_GRAD:
        return np.linalg.norm(value) < threshold
```

(6) 定义梯度下降搜索解的过程函数;

```
# 223428010210 陈梓欣
import time
#alpha 学习率
def descent(data, theta, batchSize, stopType, thresh, alpha):
    #梯度下降求解

    init_time = time.time()
    i = 0 # 迭代次数
    k = 0 # batch
    X, y = shuffleData(data)
    grad = np.zeros(theta.shape) # 计算的梯度
    costs = [cost(X, y, theta)] # 损失值

    while True:
        grad = gradient(X[k:k+batchSize], y[k:k+batchSize], theta)
        k += batchSize #取batch数量个数据
        if k >= n:
            k = 0
            X, y = shuffleData(data) #重新洗牌
            theta = theta - alpha*grad # 参数更新
            costs.append(cost(X, y, theta)) # 计算新的损失
            i += 1

            if stopType == STOP_ITER:        value = i
            elif stopType == STOP_COST:      value = costs
            elif stopType == STOP_GRAD:      value = grad
            if stopCriterion(stopType, value, thresh): break

    return theta, i-1, costs, grad, time.time() - init_time
```

(7) 调用梯度下降方法, 并绘制结果;

```
# 223428010210 陈梓欣
from sklearn.preprocessing import StandardScaler

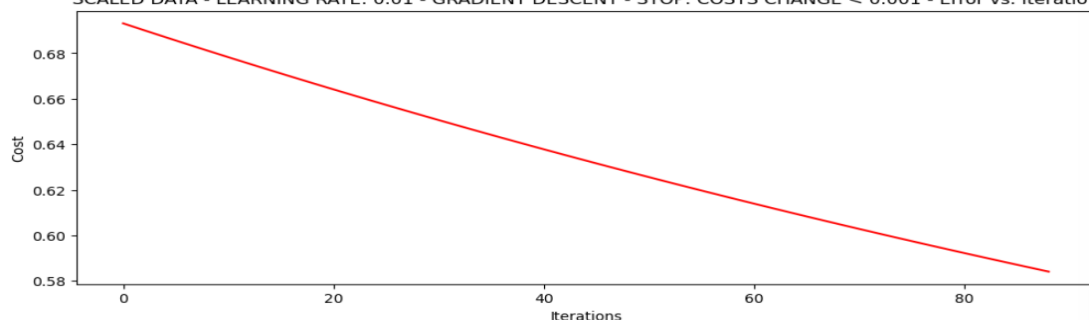
# 标准化数据
scaled_data = orig_data.copy()
scaled_data[:, 1:3] = StandardScaler().fit_transform(orig_data[:, 1:3])

# 初始化参数
theta = np.zeros([1, 3])

# 定义样本总数 n
n = scaled_data.shape[0]

# 调用梯度下降并绘制图像
theta_opt = runExpe(
    data=scaled_data,
    theta=theta,
    batchSize=n,          # 全量梯度下降
    stopType=STOP_COST,   # 成本函数停止策略
    thresh=0.001,
    alpha=0.01
)
```

```
***Scaled data - learning rate: 0.01 - Gradient descent - Stop: costs change < 0.001
Theta: [[0.07916821 0.22309856 0.19921644]] - Iter: 87 - Last cost: 0.58 - Duration: 0.01s
SCALED DATA - LEARNING RATE: 0.01 - GRADIENT DESCENT - STOP: COSTS CHANGE < 0.001 - Error vs. Iteration
```

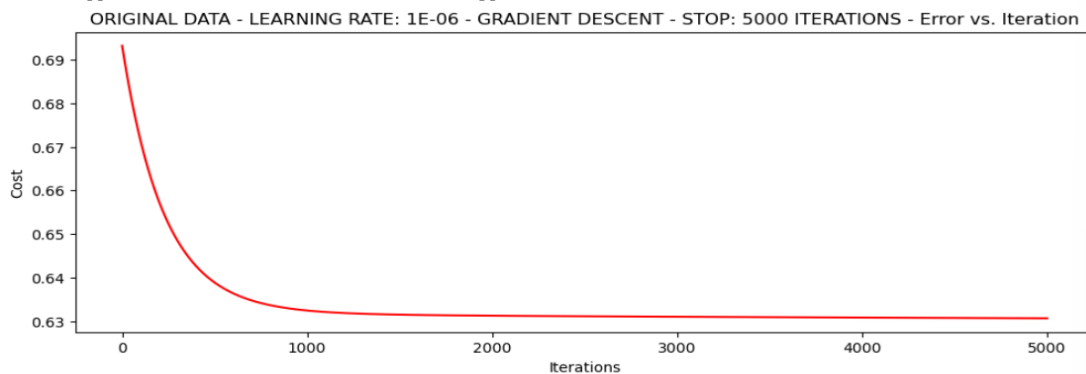


8. Logistic 回归中终止策略与目标函数值优化

(1) 应用三种不同终止策略，进行回归并给出结果；

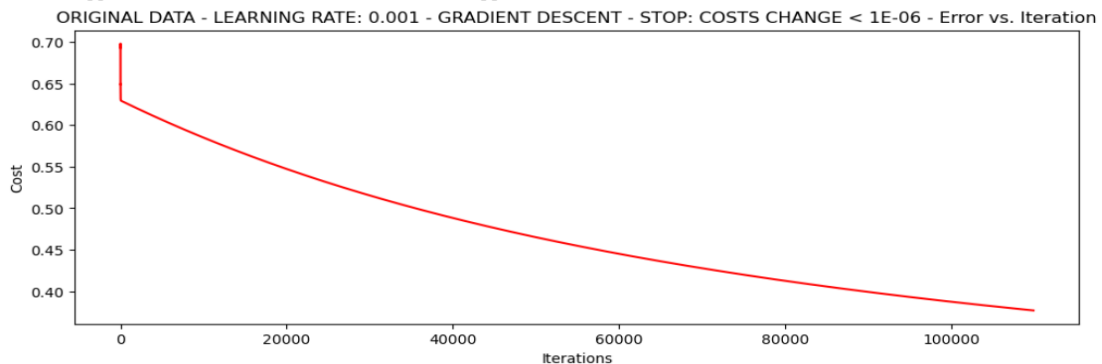
```
# 223428010210 陈梓欣
#选择的梯度下降方法是基于所有样本的
#整体进行梯度下降
n=100
runExpe(orig_data, theta, n, STOP_ITER, thresh=5000, alpha=0.000001)
```

```
***Original data - learning rate: 1e-06 - Gradient descent - Stop: 5000 iterations
Theta: [[-0.00027127  0.00705232  0.00376711]] - Iter: 5000 - Last cost: 0.63 - Duration: 1.13s
```



```
# 223428010210 陈梓欣
#根据损失值停止 设定阈值 1E-6, 差不多需要110 000次迭代
runExpe(orig_data, theta, n, STOP_COST, thresh=0.000001, alpha=0.001)
```

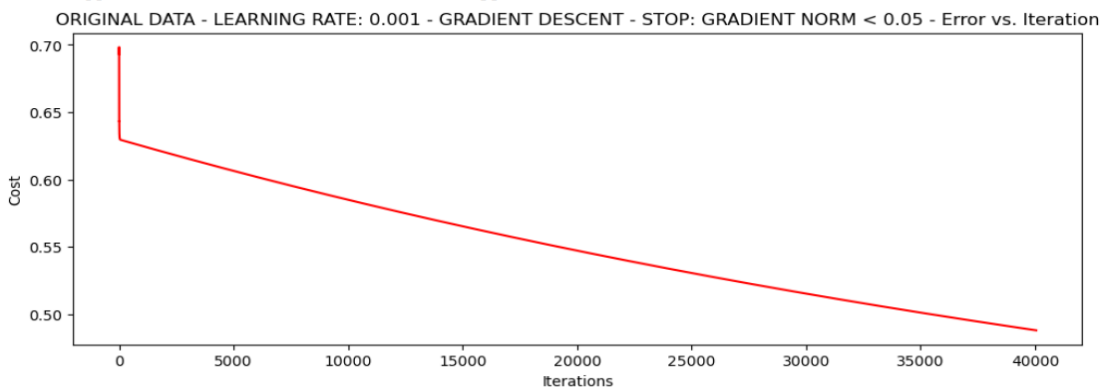
```
***Original data - learning rate: 0.001 - Gradient descent - Stop: costs change < 1e-06
Theta: [[-5.13364014  0.04771429  0.04072397]] - Iter: 109901 - Last cost: 0.38 - Duration: 12.35s
```



```
array([[-5.13364014,  0.04771429,  0.04072397]])
```

```
# 223428010210 陈梓欣
#根据梯度变化停止 设定阈值 0.05,差不多需要40 000次迭代
runExpe(orig_data, theta, n, STOP_GRAD, thresh=0.05, alpha=0.001)
```

```
***Original data - learning rate: 0.001 - Gradient descent - Stop: gradient norm < 0.05
Theta: [[-2.37033409  0.02721692  0.01899456]] - Iter: 40045 - Last cost: 0.49 - Duration: 4.55s
```

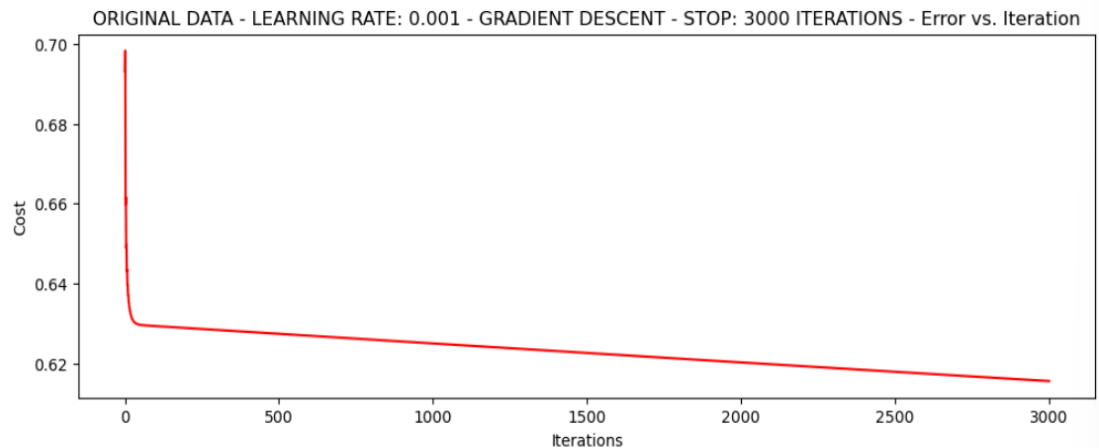


(2) 学习率对回归的影响;

```
# 223428010210 陈梓欣
# 测试不同学习率下的表现
for alpha in [0.001, 0.0001, 0.00001]:
    runExpe(orig_data, theta, batchSize=n, stopType=STOP_ITER, thresh=3000, alpha=alpha)
```

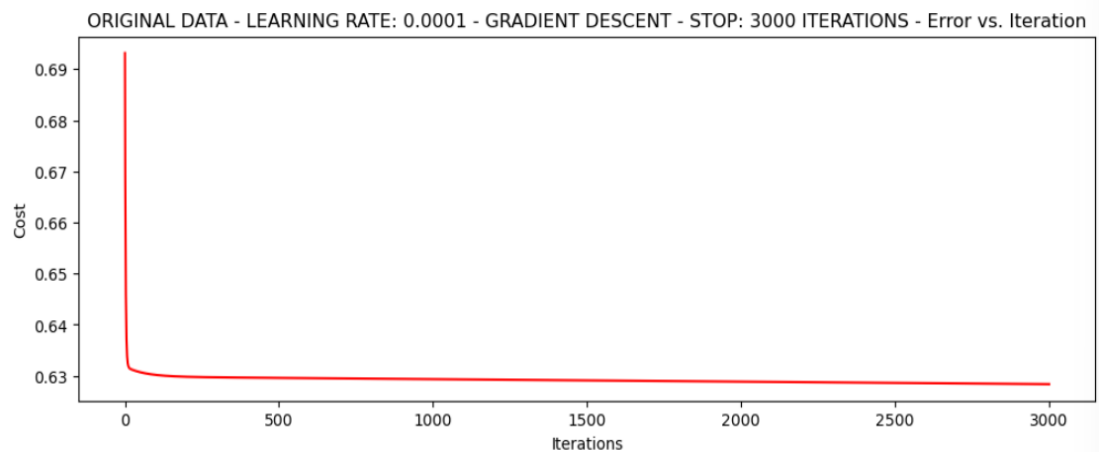
***Original data - learning rate: 0.001 - Gradient descent - Stop: 3000 iterations

Theta: $\begin{bmatrix} -0.2068016 & 0.01186208 & 0.00205968 \end{bmatrix}$ - Iter: 3000 - Last cost: 0.62 - Duration: 0.37s



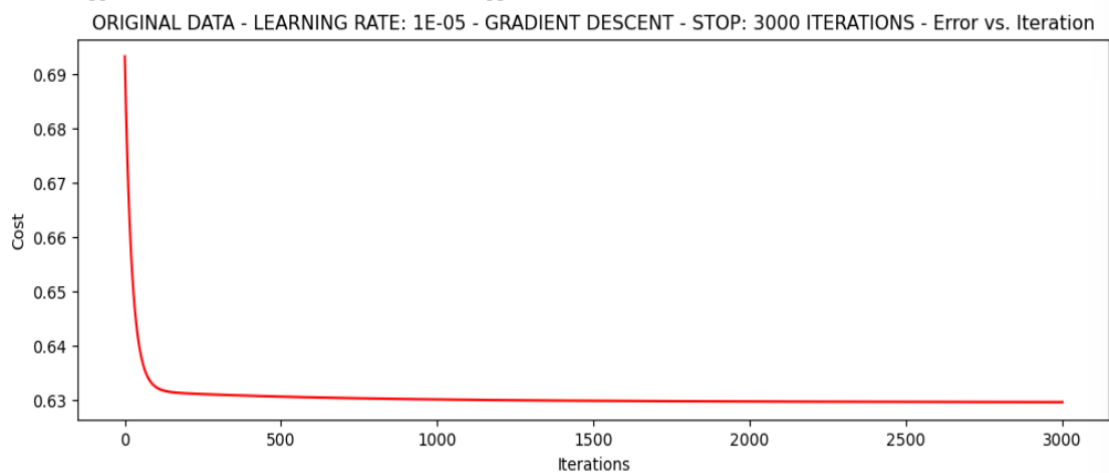
***Original data - learning rate: 0.0001 - Gradient descent - Stop: 3000 iterations

Theta: $\begin{bmatrix} -0.02085309 & 0.01056995 & 0.00061353 \end{bmatrix}$ - Iter: 3000 - Last cost: 0.63 - Duration: 0.34s

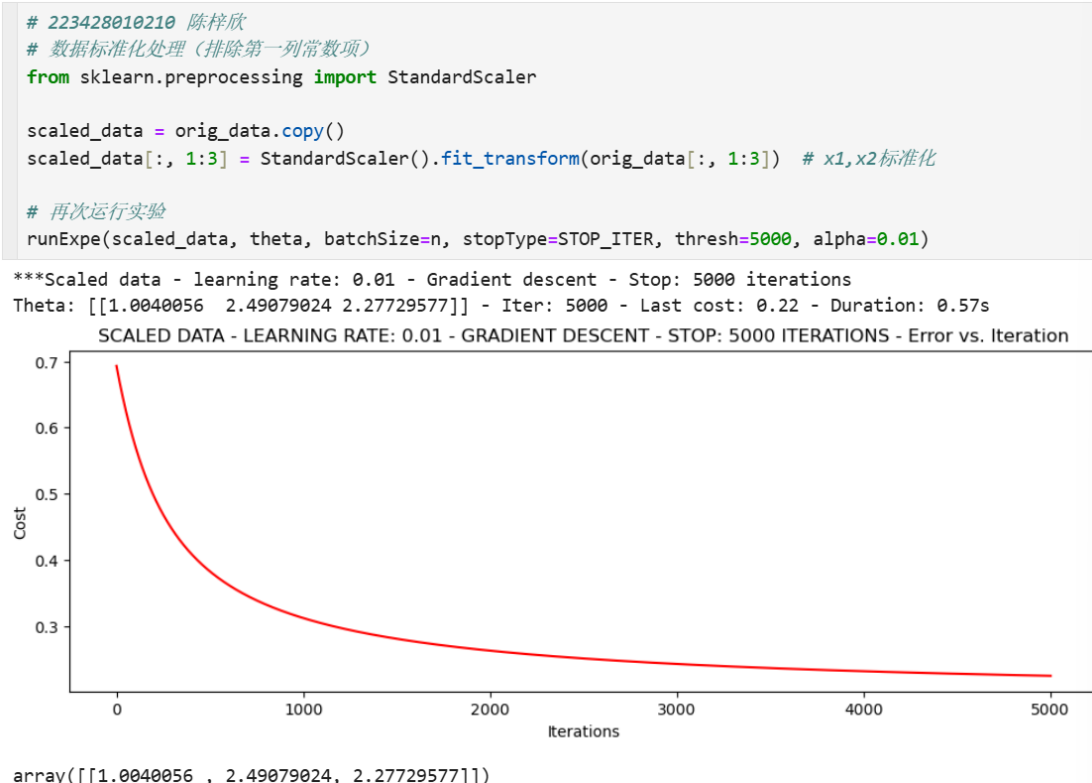


***Original data - learning rate: 1e-05 - Gradient descent - Stop: 3000 iterations

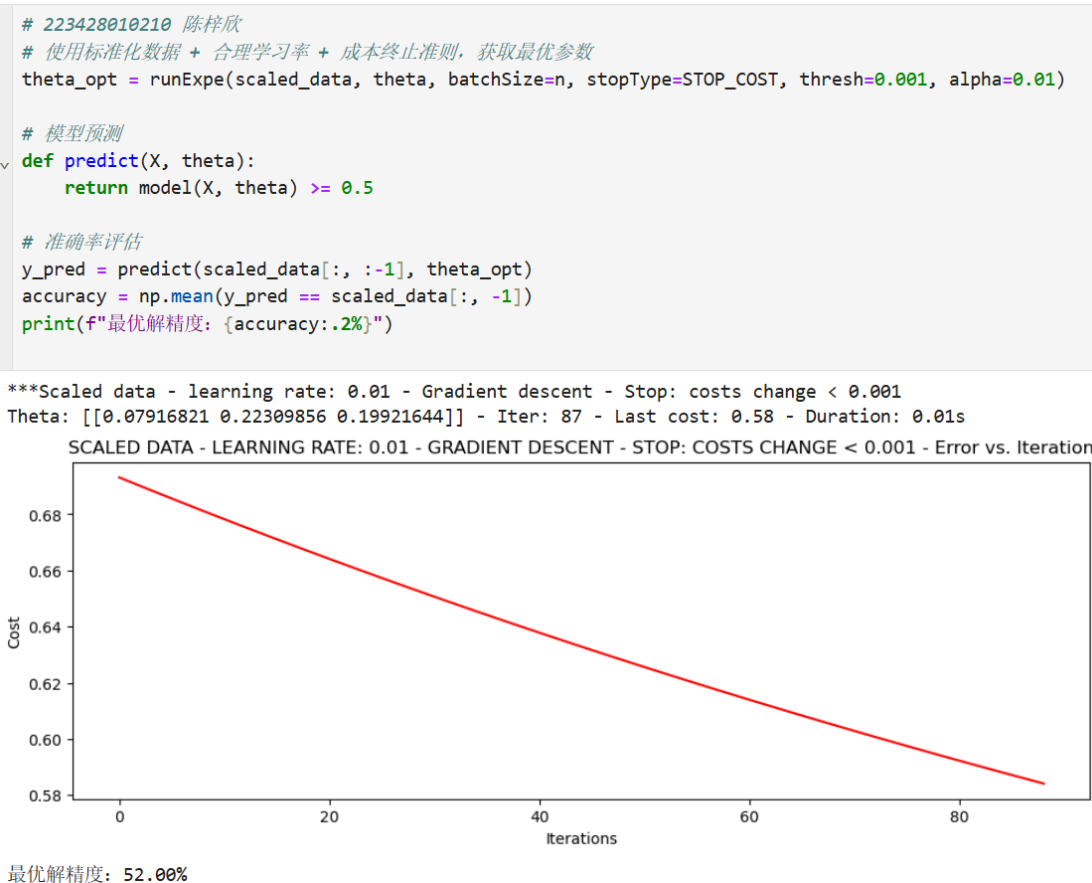
Theta: $\begin{bmatrix} -0.00201938 & 0.00998486 & 0.00091082 \end{bmatrix}$ - Iter: 3000 - Last cost: 0.63 - Duration: 0.33s



(3) 数据标准化对结果的影响;



(4) 给出最优解并求其精度;



五、实验中遇到的问题及解决方法

问题：调用梯度下降方法时报错：NameError: name 'n' is not defined。

解决方法：在运行 `runExpe()` 前，需要定义 `n` 为样本总数，因为 `n` 是用于指定 `batchSize` 的大小（全量梯度下降时 `batchSize = n`）：`n = orig_data.shape[0]` 或 `scaled_data.shape[0]`（如果用标准化数据）

问题：模型不收敛或收敛缓慢。

解决方法：适当增大学习率（如 0.01），并对数据进行标准化处理，提升收敛速度与稳定性。

六、实验总结及心得

本次实验围绕金融数据分析与 Logistic 回归模型展开，旨在提升对金融数据处理与机器学习基础方法的理解与应用能力。首先，在数据处理阶段，通过 `tushare` 获取茅台股票历史数据，并掌握了 `resample` 方法对数据按日、周、月等周期进行汇总统计，增强了对金融时间序列数据的周期性特征理解。同时，通过对 Python 时间对象的基本操作，进一步掌握了金融数据的时间索引处理技巧。

在建模部分，我们深入学习了 Logistic 回归原理及其在分类问题中的应用，理解了 `sigmoid` 函数的数学定义及其将预测值映射到概率范围的意义。随后，基于梯度下降算法，逐步优化回归模型参数。通过设置不同的停止准则（如迭代次数、梯度范数、损失变化）与学习率，观察模型收敛过程与精度表现，进一步加深了对梯度下降法本质与收敛机制的理解。

此外，我们还对原始数据进行了标准化预处理，发现其对提升模型训练效果和收敛速度具有显著作用。本次实验将数据处理与模型优化紧密结合，提升了我们将理论应用于实际问题的能力，为后续深入学习金融数据建模与机器学习方法奠定了良好基础。